

# **BÀI 1: TỔNG QUAN VỀ PHÂN TÍCH VÀ QUẢN LÝ YÊU CẦU PHẦN MỀM** (Số tiết: 5 tiết)

## **1.1. Các nguyên lý chính trong kỹ nghệ yêu cầu**

### *1.1.1 Kỹ nghệ yêu cầu*

Tương tự như kỹ nghệ phần mềm. Nói đến kỹ nghệ người ta thường đề cập đến ba khía cạnh cần nghiên cứu bao gồm: kỹ nghệ đó sử dụng quy trình nào? Quy trình đó áp dụng những phương pháp nào trong việc triển khai từng hoạt động trong quy trình? Và để tăng tính tự động, tính hỗ trợ trong quy trình, ta cần sự trợ giúp của những công cụ gì.

Do vậy, kỹ nghệ yêu cầu đề cập đến ba khía cạnh chính, nó là sự tích hợp của (1) *Quy trình*; (2) *Phương pháp*; (3) *Công cụ nhằm phát triển; đánh giá chất lượng*; và *quản lý yêu cầu phần mềm* một cách hiệu quả.

Mỗi giai đoạn trong quy trình vòng đời phần mềm đều có mục đích riêng. Quy trình phần mềm thường chia thành 4 giai đoạn chính: Phân tích, thiết kế, xây dựng, và vận hành sử dụng phần mềm tương ứng nhằm trả lời 4 câu hỏi chính WHAT, HOW, BUILD và USE. Mục đích chính của kỹ nghệ yêu cầu là trả lời câu hỏi WHAT, và các câu hỏi phụ gồm WHY, WHO và WHERE nhằm xác định rõ lý do gì yêu cầu được đề xuất, ai đề xuất, và yêu cầu đó nằm ở đây trong quy trình nghiệp vụ của khách hàng.

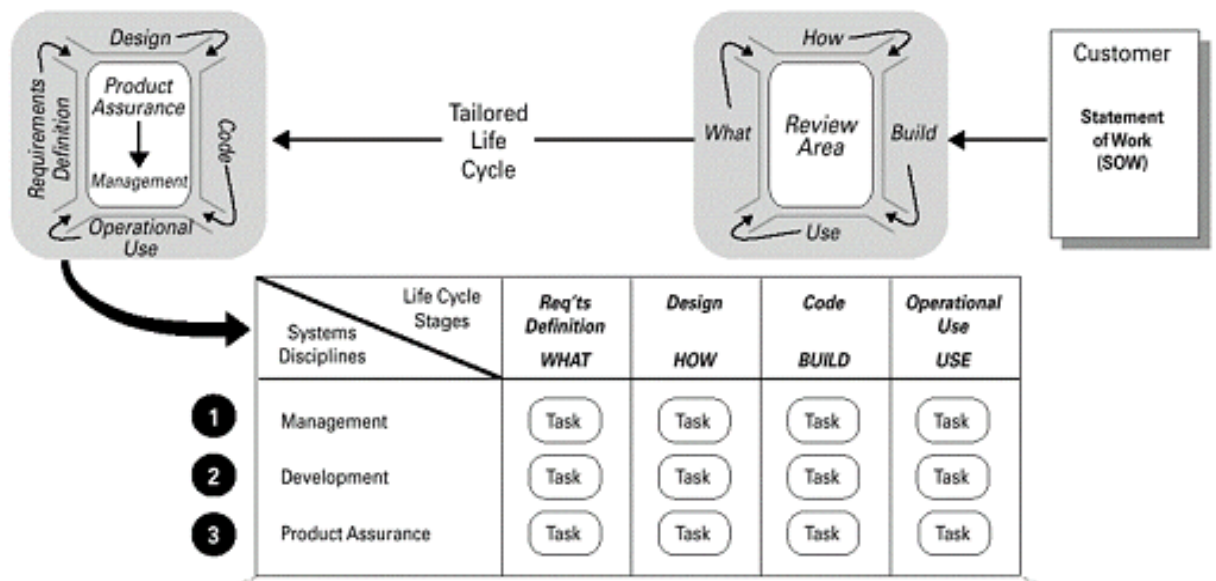
Như vậy có thể nói, phân tích yêu cầu là giai đoạn đầu tiên, có thể xuyên suốt trong vòng đời phần mềm/dự án, hoặc xuyên suốt thời gian sống của sản phẩm.

Có 3 luồng công việc/nhóm nguyên tắc/workflow chính cần thực thi trong giai đoạn này gồm:

1. Quản lý (Management);
2. Phát triển (Development) &
3. Đảm bảo chất lượng yêu cầu (Requirement Assurance).

### *1.1.2 Các nguyên lý chính trong kỹ nghệ yêu cầu*

Các nguyên tắc chính trong kỹ nghệ yêu cầu được chỉ ra trong Hình 1.1:



**Hình 1.1 Các nguyên tắc kỹ nghệ trong vòng đời phần mềm**

❑ Phát triển yêu cầu

=> Thu thập, phân tích xác định, tinh chỉnh, gom nhóm

=> Kỹ thuật trợ giúp

❑ Đảm bảo chất lượng yêu cầu

=> Đánh giá, thẩm định phát hiện các vấn đề tiềm ẩn trong yêu cầu

=> Đảm bảo chất lượng tập yêu cầu

=> Các tiêu chí thẩm định + hình thức thẩm định

❑ Quản lý yêu cầu

=> Đặc tả yêu cầu, quản lý thông tin & các thuộc tính, theo dõi dấu vết, các thay đổi yêu cầu

=> 2 cách thức quản lý: Thủ công (xây dựng các tài liệu) & Tự động (dùng Tools)

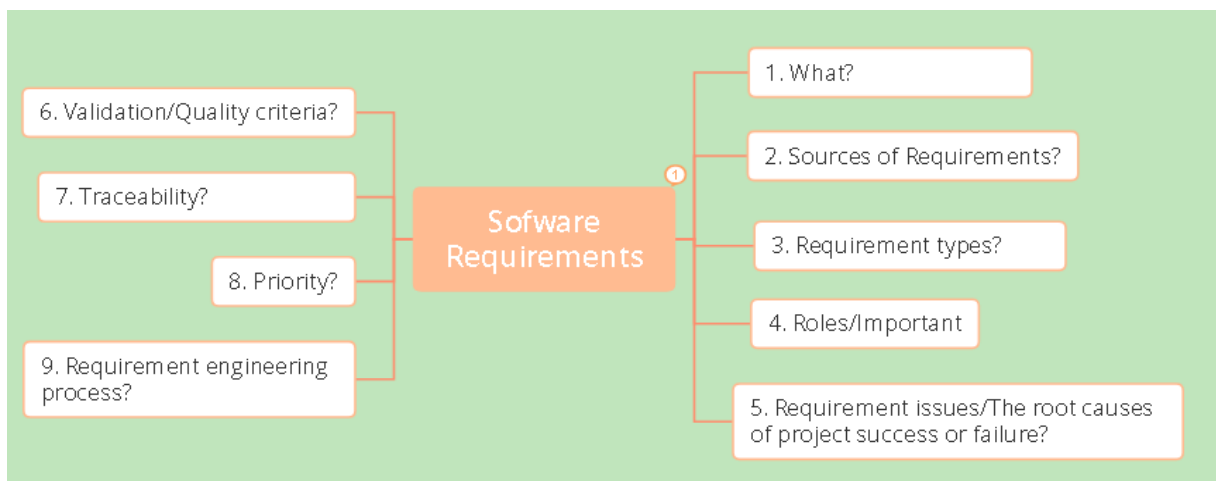
Các nguyên tắc này có thể tách thành các bước nhỏ hơn như chỉ ra trong Hình 1.2



**Hình 1.2 Các bước trong quy trình kỹ nghệ yêu cầu**

## 1.2. Yêu cầu phần mềm và các thông tin liên quan

Hình sau đây là những nội dung chúng ta sẽ tìm hiểu trong mục này:



**Hình 1.3 Yêu cầu phần mềm và các thông tin liên quan**

Chúng bao gồm:

- Yêu cầu phần mềm là gì?
- Nguồn gốc của các yêu cầu phần mềm?
- Phân loại yêu cầu phần mềm?
- Vai trò/tầm quan trọng của yêu cầu phần mềm?
- Các vấn đề tiềm ẩn trong yêu cầu phần mềm là gì? Đây là những nguyên nhân gốc rễ làm cho dự án phần mềm trở nên thách thức và có thể dẫn đến thất bại liên quan đến yêu cầu phần mềm?
- Làm thế nào để xác định các vấn đề tiềm ẩn trong các phát biểu yêu cầu? Và các tiêu chí nào cần thiết cho việc rà soát để phát hiện các vấn đề này?
- Sau khi thu được yêu cầu dạng thô từ các bên liên quan, phân tích viên cần tiếp tục xử lý bằng cách tinh chỉnh, làm rõ, phân tách, gom nhóm và xác định các

thông tin liên quan cần quản lý của yêu cầu. Tất cả những việc tinh chỉnh, tách, gộp, ... đều cần phải quản lý được dấu vết giữa chúng. Vậy quản lý và theo dõi dấu vết giữa các yêu cầu đóng vai trò quan trọng như thế nào?

- Để gán độ ưu tiên cho các yêu cầu nhằm trợ giúp cho hoạt động lập kế hoạch triển khai, ta có thể dùng phương pháp nào để gán nhằm đạt được sự thống nhất giữa các bên liên quan?
- Quy trình kỹ nghệ yêu cầu có thể tách thành quy trình phát triển & thẩm định yêu cầu; quy trình quản lý yêu cầu. Vậy cụ thể từng hoạt động trong mỗi quy trình này ra sao? Nếu dùng cách tiếp cận cụ thể ví dụ mô hình kim tự tháp yêu cầu thì các hoạt động này sẽ được tùy biến như thế nào?

### 1.2.1 Yêu cầu phần mềm

Yêu cầu phần mềm là phát biểu/mô tả về một *dịch vụ* mà p.mềm phải cung cấp; 1 *mục tiêu* mà p.m phải đạt được; 1 *ràng buộc* p.m phải tuân theo, hoặc 1 *điều kiện/phụ thuộc/giả định* mà p.m phải thỏa mãn; **nhu cầu**, (**mong muốn**) của stakeholder; *thuộc tính chất lượng* mà p.mềm phải có.

#### **Lưu ý:**

“Requirements only described the “what” of a system, not the “how”

Luôn phải nhớ: “có một sự khác biệt lớn giữa mong muốn và nhu cầu thiết yếu”

#### **Ví dụ:**



**Hình 1.4 Sự khác biệt giữa “needs” & “wants”**

Để xác định needs, bạn cần luôn giữ trong đầu câu hỏi: ***“Tôi có thực sự cần những thứ này không?”***



Hình 1.5 Các thức xác định đâu là needs

⇒ Trong Hình 1.5, để sửa một chiếc xe đạp, trong hộp đồ nghề của bạn sẽ không cần phải có giao, kéo, thước kẻ hoặc áp phích, ...

### Thảo luận:

- Needs & wants của bạn về vấn đề ăn, mặc, ở, tài sản là gì?
- Sự khác biệt giữa needs & wants là gì?

### 1.2.2 Nguồn gốc của yêu cầu phần mềm

#### **Yêu cầu phần mềm đến từ nhiều nguồn:**

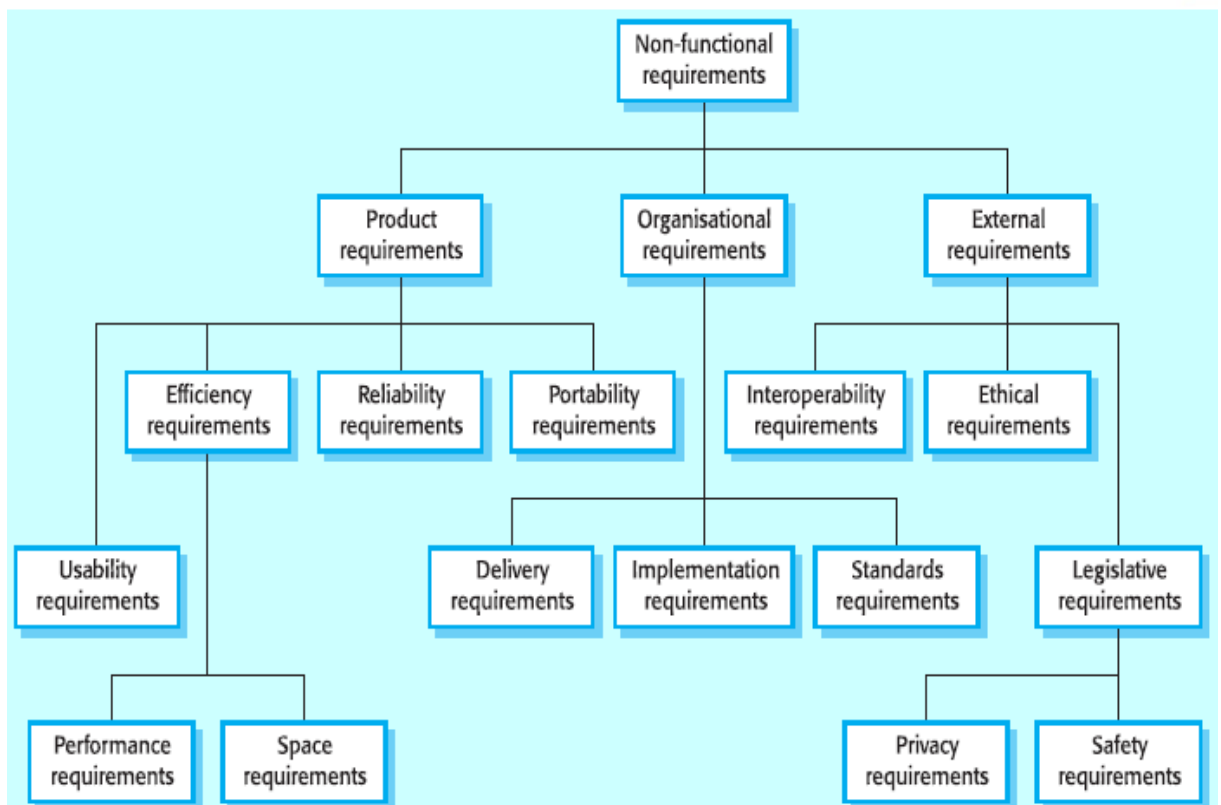
- Các bên liên quan (Stakeholders)
- Những người bị tác động theo một cách nào đó bởi hệ thống
- đối tượng chính: **khách hàng** (đầu tư, ví dụ: tổ chức, đầu giá trò chơi) & **người dùng cuối** (sử dụng, ví dụ: người xem, người chơi)
- Các tài liệu (Documents)
- Hệ thống đang tồn tại (Existing system)
- Miền ứng dụng (Application Domain)
- Mục tiêu của phần mềm (the goal of the software)

### 1.2.3 Phân loại yêu cầu

Nếu phân loại yêu cầu theo mức độ trừu tượng của đặc tả, và theo đối tượng người đọc yêu cầu ta có thể chia thành hai loại:

- Yêu cầu người dùng (Use Requirements):Viết cho stakeholders đọc

- Yêu cầu hệ thống (System Requirements): Viết cho developers, testers, QA đọc  
Yêu cầu người dùng có thể tiếp tục được phân thành các loại nhỏ hơn gồm:
  - Yêu cầu chức năng (Functional Requirements)
    - ⇒ Phản chức năng/dịch vụ cụ thể của phần mềm
    - Ví dụ: *“Hệ thống sẽ gửi email để xác nhận giao dịch đặt trước của khách hàng”*
  - Yêu cầu phi chức năng (Nonfunctional Requirements)
    - ⇒ Không trực tiếp phản ánh các chức năng của phần mềm, nhưng phần mềm phải thoả mãn thì nó mới sử dụng được.
    - Ví dụ: *“Phần mềm an toàn khi sử dụng”*; *“Phần mềm dễ sử dụng”*
    - ⇒ Cần phát biểu một cách có định lượng để dễ cho việc kiểm thử, nghiệm thu
    - Ví dụ: *“Phần mềm dễ sử dụng” => “Thời gian huấn luyện người dùng không quá 2 tuần, số lỗi mắc phải không quá 2 lỗi/ngày”*
    - ⇒ Yêu cầu phi chức năng có thể xuất phát từ các nhu cầu của tổ chức người dùng, từ môi trường vận hành sản phẩm, từ các thuộc tính chất lượng. Do đó chúng có thể được chia thành 3 loại theo nguồn gốc xuất xứ như chỉ ra trong Hình 1.6



**Hình 1.6 Phân loại các yêu cầu phi chức năng**

- Yêu cầu miền (Domain Requirements)

⇒ Chứa các thuật ngữ, khái niệm đặc thù của miền ứng dụng, yêu cầu BAs phải hiểu một cách rõ ràng

- Có thể là yêu cầu chức năng, phi chức năng
- Ví dụ:

*“The system’s GUI complies the Z39.50 library standard.”*

*“The deceleration/giảm tốc of the train shall be computed as:*

$$D(\text{train}) = D(\text{control}) + D(\text{gradient})$$

*Where:  $D(\text{gradient})$  is  $9.81\text{ms}^2 * \text{compensated gradient}/\alpha$  and the values of  $9.81\text{ms}^2 / \alpha$  are known for different types of train”.*

- Yêu cầu đảo (Reverse Requirements)

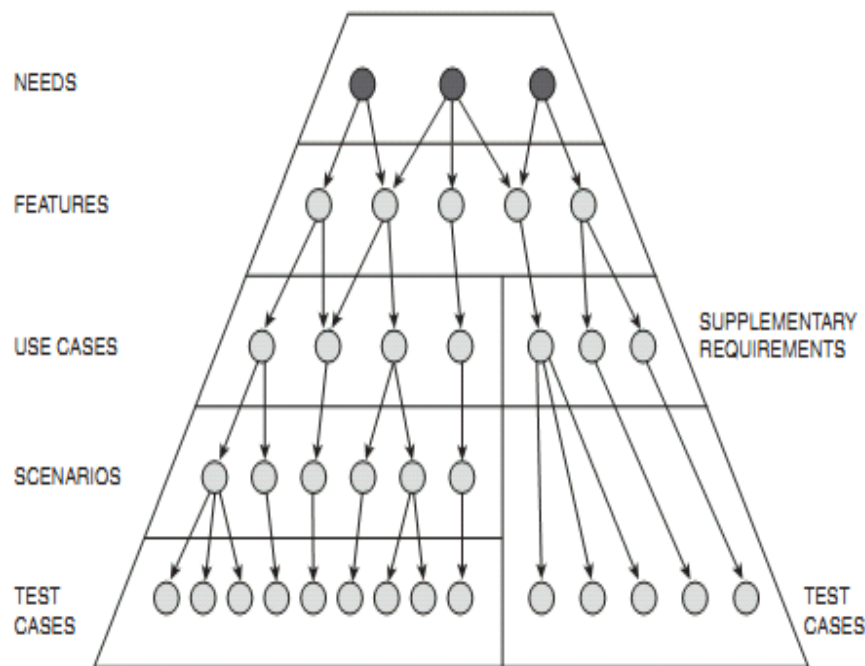
Yêu cầu đảo giải thích những gì hệ thống sẽ không làm; hoặc mô tả các ràng buộc trên các hành vi được phép. Một số yêu cầu thích hợp phát biểu dưới dạng yêu cầu đảo, ví dụ: *“Hệ thống sẽ không sử dụng màu đỏ trong giao diện người dùng bất cứ khi nào yêu cầu họ nhập thông tin đầu vào”*. Hoặc các yêu cầu về an toàn và an ninh cũng thường được phát biểu theo dạng này. Ví dụ: *“Hệ thống không cho phép những người dùng không có thẩm quyền truy cập các chức năng quản trị”*; *“Hệ thống không công khai các thông tin cá nhân của người dùng”*

Theo mô hình kim tự tháp yêu cầu có thể chia thành các loại:

- NEEDs (Stakeholder’s Requests)
- Product Features (FEATs)
- Use cases (Ucs)
- Supplementary requirements (SUPLs)

Trong đó: *Needs, Feats (~ user requirements); Ucs, SUPLs (system requirements).*

Hình 1.7 chỉ ra mô hình kim tự tháp yêu cầu.



**Hình 1.7 Mô hình kim tự tháp yêu cầu**

#### 1.2.4 Vai trò, tầm quan trọng của yêu cầu

Yêu cầu phần mềm là đối tượng nghiên cứu chính của kỹ nghệ yêu cầu. Chúng đóng vai trò chính như sau:

- Là cơ sở nền tảng cho các công việc phát triển hệ thống sau đó như hoạt động thiết kế hệ thống, phát triển, kiểm thử, thực thi và vận hành, ...
- Là cơ sở cho việc mời thầu. Do đó, các yêu cầu cần phát biểu sao cho ngắn gọn, rõ ràng, dễ hiểu
- Là cơ sở ký kết hợp đồng thầu, nên chúng cần đủ chi tiết, chính xác để tránh hiểu sai, hiểu nhầm
- Làm đầu vào cho thiết kế và triển khai và là cơ sở để lập kế hoạch triển khai dự án; quản lý rủi ro; kiểm thử chấp thuận; quản lý thay đổi, quản lý cấu hình và quản lý các phiên bản phần mềm.

#### 1.2.5 Các vấn đề tiềm ẩn trong yêu cầu

Nhiều dự án thất bại liên quan đến yêu cầu & quản lý yêu cầu nghèo nàn, không được coi trọng. Các vấn đề liên quan đến yêu cầu có thể xuất phát từ việc giao tiếp, truyền thông không hiệu quả, trong đặc tả yêu cầu, trong quản lý yêu cầu.

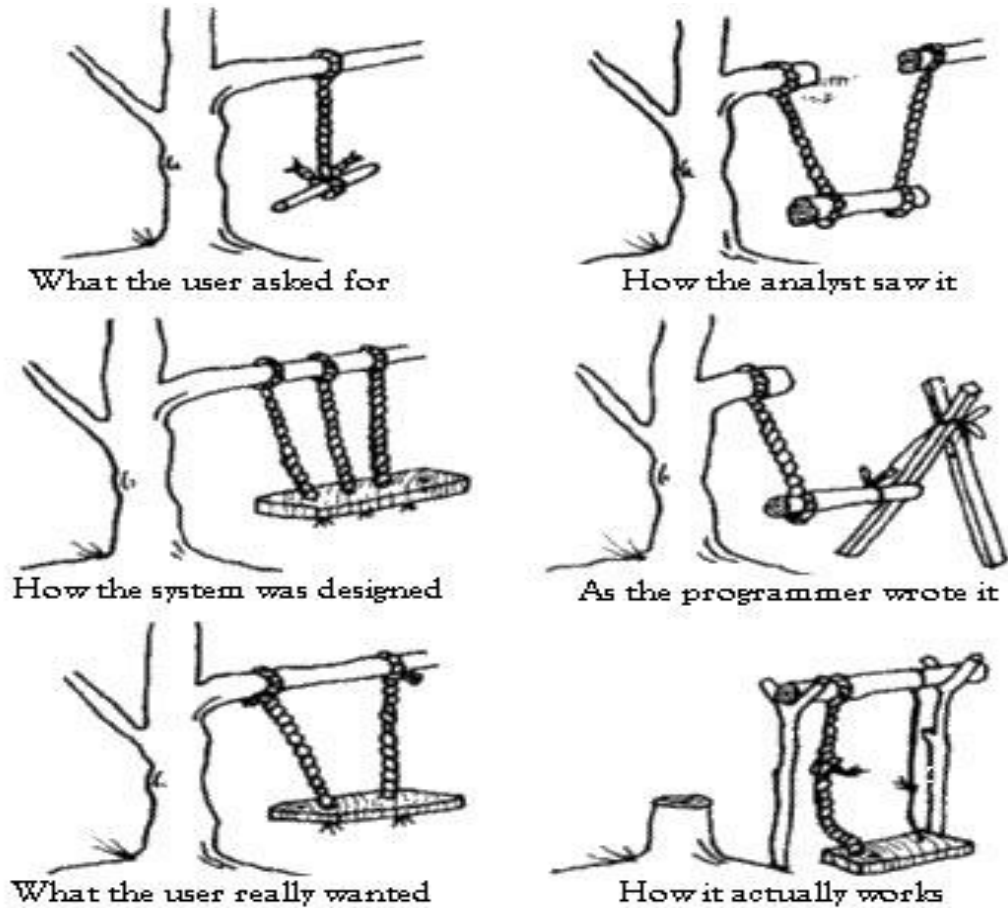
Ví dụ: truyền thông, giao tiếp không hiệu quả dẫn đến hiểu sai, hiểu nhầm:



© Scott Adams, Inc./Dist. by UFS, Inc.

Hình 1.8 Ví dụ về giao tiếp không hiệu quả

Đặc tả không rõ ràng, dẫn đến hiểu sai hiểu nhầm giữa các bên liên quan:



### Hình 1.9 Ví dụ về hậu quả truyền thông, đặc tả yêu cầu không hiệu quả

Phát biểu yêu cầu (what) không rõ ràng dẫn hiểu sai, triển khai sai và phần mềm tạo rủi ro cho người dùng khi vận hành.



Hình 1.10 Ví dụ về hậu quả phát biểu yêu cầu không rõ ràng

Khi thu thập yêu cầu (What) không xác định (Why), tức lý do tại sao người dùng cần yêu cầu này, dẫn đến phần mềm không thực sự có giá trị và ý nghĩa thiết thực và tiềm ẩn những rủi ro khi sử dụng:



Hình 1.11 Ví dụ về thu thập yêu cầu không hiểu rõ lý do.

#### 1.2.6 Thẩm định, đánh giá yêu cầu

Thẩm định hay còn gọi là đánh giá yêu cầu nhằm phát hiện và giải quyết sớm các vấn đề về yêu cầu, nhằm đảm bảo các yêu cầu thu thập, phân tích xác định là đảm bảo chất lượng và không tiềm ẩn những vấn đề bên trong nó.

Để thẩm định, chúng ta thường dựa trên các tiêu chí đánh giá như sau:

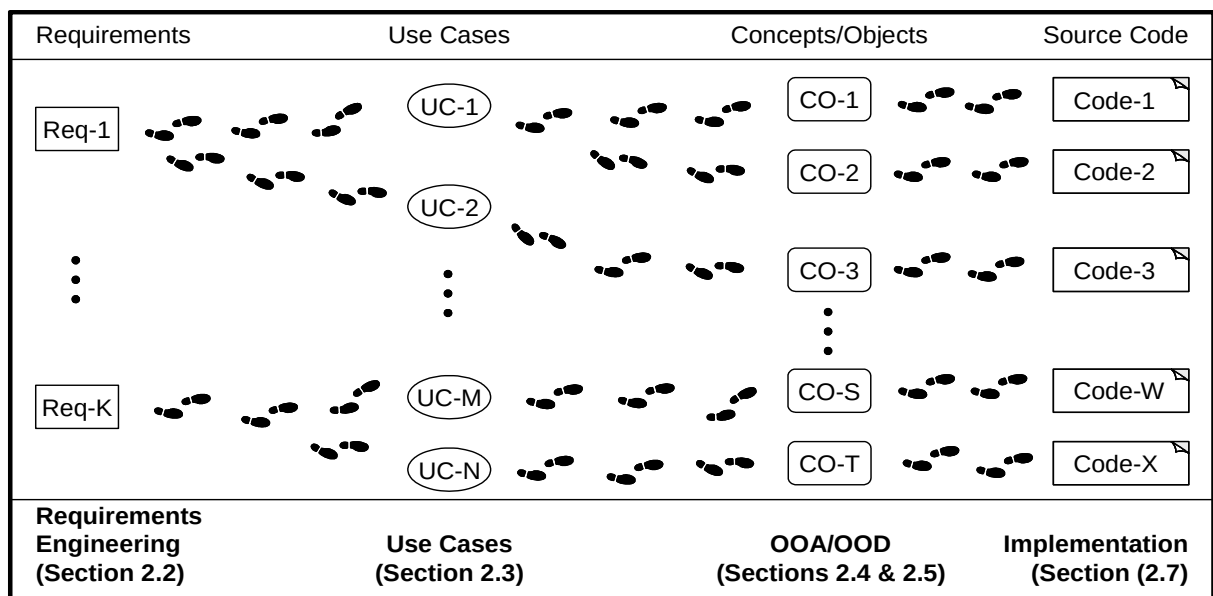
- Các tiêu chí đánh giá từng yêu cầu riêng lẻ
  - ✓ Không mập mờ (Unambiguous)
  - ✓ Đúng đắn (Correct)
  - ✓ Nguyên tử (Atomic)
  - ✓ Khả thi (Feasibility)
  - ✓ Có khả năng kiểm thử (Testable)

- ✓ Cô đọng (Cohesive)
  - ✓ Cần thiết (Necessary)
  - ✓ Có thể hiểu (Under-standable)
  - ✓ Độc lập (Independent)
  - ✓ Đầy đủ (Complete)
  - ✓ Có khả năng theo dõi (Traceable).
- Các tiêu chí đánh giá tập yêu cầu
    - ✓ Thống nhất (Consistent)
    - ✓ Không dư thừa (Non-redundant)
    - ✓ Đầy đủ (Complete)

### 1.2.7 Theo dõi dấu vết yêu cầu

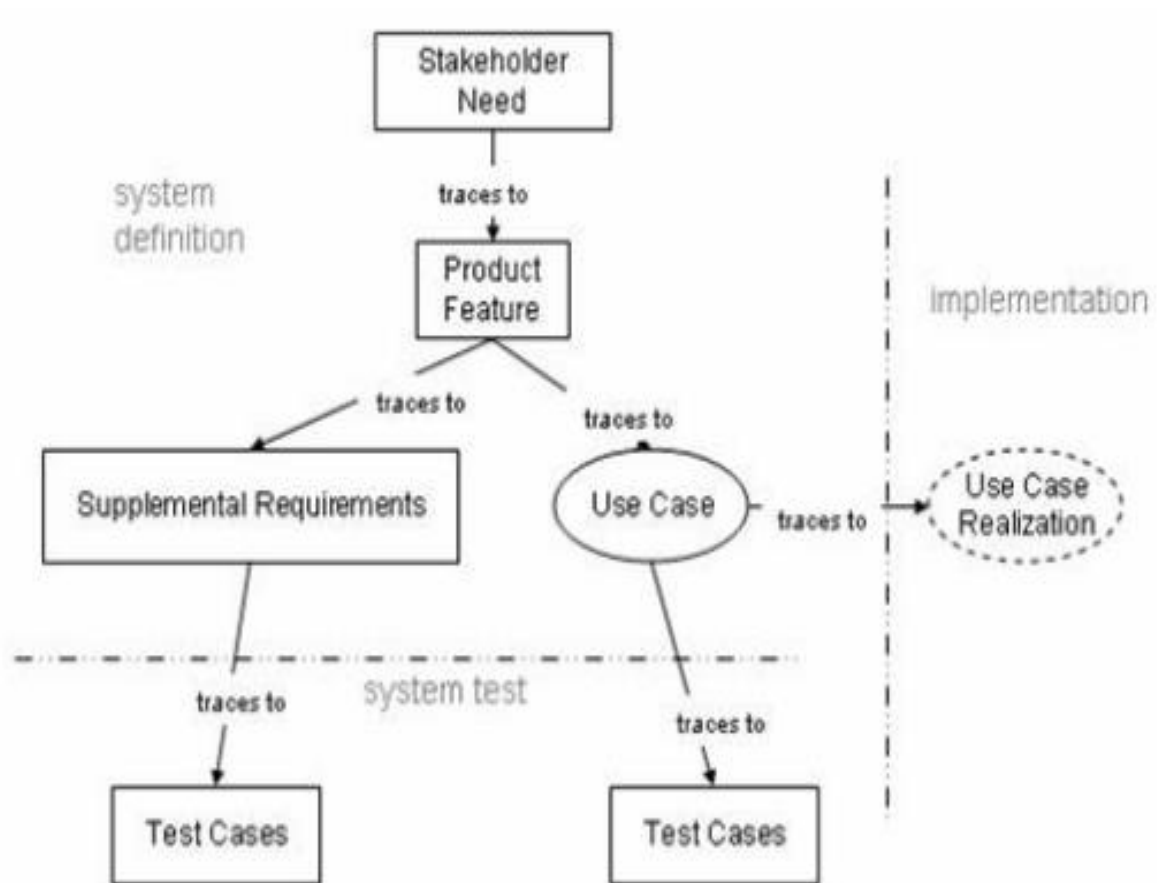
Theo dõi dấu vết giữa các thành phẩm dự án là quan trọng vì nhờ đó ta có thể đảm bảo sự thống nhất giữa các thành phần này, đặc biệt giữa những loại yêu cầu sau khi được phân tích, tinh chỉnh và đặc tả. Bên cạnh đó, giúp ta tìm các nguyên nhân của một vấn đề tồn tại một cách dễ dàng.

Hình 1.12 chỉ ra dấu vết giữa các thành phẩm dự án ở các giai đoạn phát triển khác nhau.



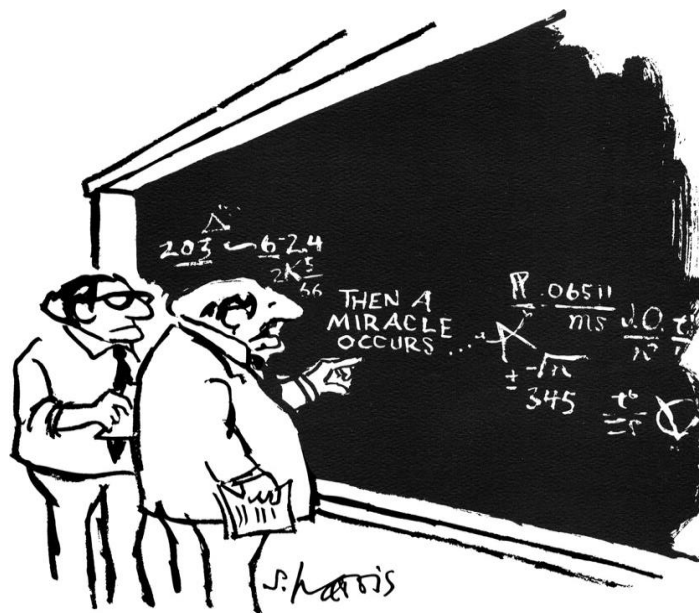
**Hình 1.12 Dấu vết giữa các thành phần dự án qua các giai đoạn phát triển**

Hình 1.13 chỉ ra dấu vết giữa các loại yêu cầu sau khi phân tích & phát triển.



Hình 1.13 Dấu vết giữa các kiểu yêu cầu phần mềm

Quản lý dấu vết giúp ta tránh những bước nhảy cóc mà không thể giải thích được. Ví dụ: “...where did this come from?” như chỉ ra trong Hình 1.14:



"I THINK YOU SHOULD BE MORE EXPLICIT HERE IN STEP TWO."

Hình 1.14 Quản lý dấu vết giúp xác định nguyên nhân của vấn đề tồn tại

Do vậy, có thể nói “*Truy xuất nguồn gốc là điều kiện tiên quyết để chứng minh sự tuân thủ.*”

#### 1.2.8 Độ ưu tiên của các yêu cầu

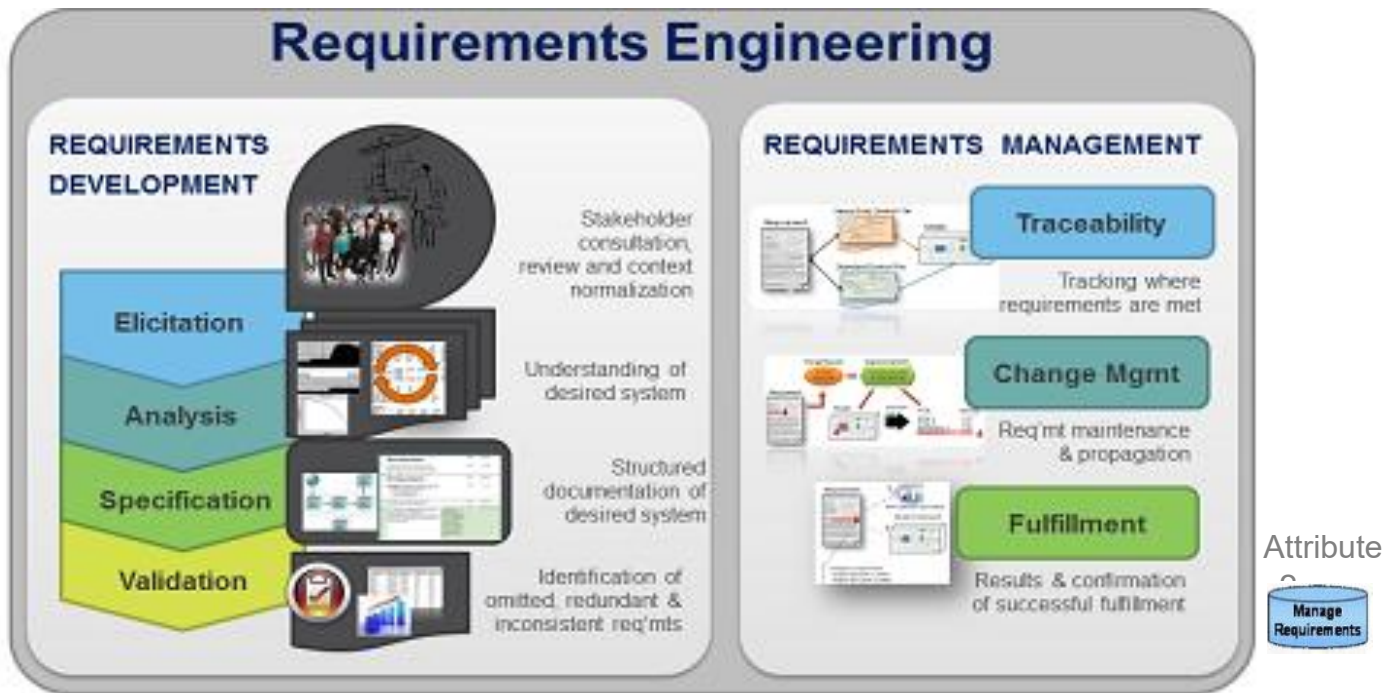
Các giá trị ưu tiên có thể nhận: **High (cao)**, **Medium (trung bình)**, **Low (thấp)**, hoặc ta có thể sử dụng phương pháp MoSCoW. MoSCoW là kỹ thuật ưu tiên hóa được sử dụng trong quản lý dự án và phát triển phần mềm để phân loại các yêu cầu dựa trên tầm quan trọng của chúng (mức độ quan trọng và sự tác động của chúng lên sự thành công của dự án) Giúp stakeholders và các teams dự án làm rõ và thống nhất về mức độ ưu tiên của các yêu cầu

#### Phương pháp MoSCoW:

- **Must Have:** Các yêu cầu rất quan trọng đối với sự thành công của dự án. Nếu các yêu cầu này không được đáp ứng, dự án có khả năng thất bại hoặc không đạt được mục tiêu.
- **Should Have:** được coi là thiết yếu nhưng không quan trọng đối với sự thành công ngay lập tức của dự án. Những yêu cầu này nên được đưa vào nếu có thể
- **Could Have:** là mong muốn nhưng không cần thiết cho sự thành công của dự án; có thì tốt; có thể xem xét nếu thời gian và nguồn lực cho phép.
- **Won't Have:** Các yêu cầu mà các bên liên quan đã đồng ý loại trừ khỏi phạm vi hiện tại của dự án. Các yêu cầu này có thể được xem xét cho các giai đoạn hoặc bản phát hành trong tương lai nhưng không được đưa vào các sản phẩm hiện tại.

#### 1.2.9 Quy trình kỹ nghệ yêu cầu

Kỹ nghệ yêu cầu chia thành 3 nhóm nguyên tắc chính gồm: (1) Phát triển tập yêu cầu; (2) đảm bảo chất lượng tập yêu cầu; (3) quản lý các yêu cầu. Trong đó (2) được kết hợp vào các hoạt động của (1) và (3) trong quy trình kỹ nghệ như chỉ ra trong Hình 1.15



Hình 1.15 Phát triển & Quản lý yêu cầu.

### 1.3. Thảo luận

Các vai trò của phân tích viên nghiệp vụ trong thu thập, phân tích và quản lý yêu cầu phần mềm?